

LAB 4 REPORT

by Vic Grigoryev (vgrigory)

(#1) Build a modified kernel

Approach: Find a place to insert an @ symbol when the kernel is idle, then write in the change.

Problems Encountered: I went into the /usr/src/ directory and was immediately lost as to where I could write in the change.

Solution: I decided that the best way to figure out the operating system was to actually look into the files and see what they do.

```
/*-----*/
$ ls
Makefile      debug.c      interrupt.c  proc.c      system      vm.h
arch          debug.h      ipc.h       proc.h      system.c    watchdog.c
clock.c       extract-errno.sh kernel.h    profile.c   system.h    watchdog.h
clock.h       extract-mfield.sh main.c      profile.h   table.c
config.h      extract-mtype.sh perf.h      proto.h     type.h
const.h       glo.h        priv.h      start.c     utility.c
$ _
```

From this list, I thought that the kernel.h file would have something related to what prints on the screen. I used vim to view it, but...

```
#ifndef KERNEL_H
#define KERNEL_H

/* APIC is turned on by default */
#define CONFIG_APIC
/* boot verbose */
#define CONFIG_BOOT_VERBOSE
/*
 * compile in the nmi watchdog by default. It is not enabled until watchdog=1
 * (non-zero) is set in monitor
 */
#define CONFIG_WATCHDOG
/* We only support 1 cpu now */
#define CONFIG_MAX_CPUS 1
#define cpuid          0

/* OXPCIe952 PCIe with 2 UARTs in-kernel support */
#define CONFIG_OXPCIE 0

/* This is the master header for the kernel. It includes some other files
 * and defines the principal constants.
 */
#define _POSIX_SOURCE      1 /* tell headers to include POSIX stuff */
#define _MINIX              1 /* tell headers to include MINIX stuff */
"kernel.h" [READONLY] 58 lines, 2083 chars
```

There wasn't anything useful in here that I could discern. I moved onto config.h, since maybe that prints things to the screen...

```

#ifndef CONFIG_H
#define CONFIG_H

/* This file defines the kernel configuration. It allows to set sizes of some
 * kernel buffers and to enable or disable debugging code, timing features,
 * and individual kernel calls.
 *
 * Changes:
 *   Jul 11, 2005          Created. (Jorrit N. Herder)
 */

/* In embedded and sensor applications, not all the kernel calls may be
 * needed. In this section you can specify which kernel calls are needed
 * and which are not. The code for unneeded kernel calls is not included in
 * the system binary, making it smaller. If you are not sure, it is best
 * to keep all kernel calls enabled.
 */
#define USE_FORK            1    /* fork a new process */
#define USE_NEWMAP          1    /* set a new memory map */
#define USE_EXEC            1    /* update process after execute */
#define USE_CLEAR           1    /* clean up after process exit */
#define USE_EXIT            1    /* a system process wants to exit */
#define USE_TRACE           1    /* process information and tracing */
#define USE_GETKSIG         1    /* retrieve pending kernel signals */
"config.h" [READONLY] 71 lines, 3421 chars

```

Same issue here. These were header files, so maybe this kind of thing wasn't what I was supposed to look for. The lab instructions did mention adding in an if statement, so I moved onto looking for C files. The first one I thought would work was main...

```

/* This file contains the main program of MINIX as well as its shutdown code.
 * The routine main() initializes the system and starts the ball rolling by
 * setting up the process table, interrupt vectors, and scheduling each task
 * to run to initialize itself.
 * The routine shutdown() does the opposite and brings down MINIX.
 *
 * The entries into this file are:
 *   main:             MINIX main program
 *   prepare_shutdown: prepare to take MINIX down
 */
#include "kernel.h"
#include <string.h>
#include <unistd.h>
#include <assert.h>
#include <a.out.h>
#include <minix/com.h>
#include <minix/endpoint.h>
#include <minix/u64.h>
#include "proc.h"
#include "debug.h"
#include "clock.h"
#include "hw_intr.h"

/* Prototype declarations for PRIVATE functions. */
"main.c" [READONLY] 331 lines, 11946 chars

```

As written on the top of the file, this is how Minix starts up and shuts down. I don't need to modify something to this degree, so I took a look into system.c...

```

/* This task handles the interface between the kernel and user-level servers.
 * System services can be accessed by doing a system call. System calls are
 * transformed into request messages, which are handled by this task. By
 * convention, a sys_call() is transformed in a SYS_CALL request message that
 * is handled in a function named do_call().
 *
 * A private call vector is used to map all system calls to the functions that
 * handle them. The actual handler functions are contained in separate files
 * to keep this file clean. The call vector is used in the system task's main
 * loop to handle all incoming requests.
 *
 * In addition to the main sys_task() entry point, which starts the main loop,
 * there are several other minor entry points:
 *   get_priv:      assign privilege structure to user or system process
 *   set_sendto_bit: allow a process to send messages to a new target
 *   unset_sendto_bit: disallow a process from sending messages to a target
 *   fill_sendto_mask: fill the target mask of a given process
 *   send_sig:      send a signal directly to a system process
 *   cause_sig:      take action to cause a signal to occur via a signal mgr
 *   sig_delay_done: tell PM that a process is not sending
 *   umap_bios:      map virtual address in BIOS_SEG to physical
 *   get_randomness: accumulate randomness in a buffer
 *   clear_endpoint: remove a process' ability to send and receive messages
 *   sched_proc:     schedule a process
"system.c" [READONLY] 667 lines, 25447 chars

```

But it turns out that this wasn't what I was looking for either. I don't need to mess with the messages the user processes and kernel send to each other.

Speaking of processes, since we're looking to change what the kernel prints when it has nothing to do, I decided to check proc.c since it seemed like a process C file.

```

/* This file contains essentially all of the process and message handling$
 * Together with "mpx.s" it forms the lowest layer of the MINIX kernel.
 * There is one entry point from the outside:
 *
 *   sys_call:      a system call, i.e., the kernel is trapped with an INT
 *
 * Changes:
 *   Aug 19, 2005    rewrote scheduling code (Jorrit N. Herder)
 *   Jul 25, 2005    rewrote system call handling (Jorrit N. Herder)
 *   May 26, 2005    rewrote message passing functions (Jorrit N. Herder)
 *   May 24, 2005    new notification system call (Jorrit N. Herder)
 *   Oct 28, 2004    nonblocking send and receive calls (Jorrit N. Herder)
 *
 * The code here is critical to make everything work and is important for the
 * overall performance of the system. A large fraction of the code deals with
 * list manipulation. To make this both easy to understand and fast to execute
 * pointer pointers are used throughout the code. Pointer pointers prevent
 * exceptions for the head or tail of a linked list.
 *
 *   node_t *queue, *new_node; // assume these as global variables
 *   node_t **xpp = &queue;    // get pointer pointer to head of queue
 *   while (*xpp != NULL)       // find last pointer of the linked list
 *       xpp = &(*xpp)->next;    // get pointer to next pointer
 *   *xpp = new_node;           // now replace the end (the NULL pointer)

```

I scrolled down, and lo and behold...

```

        break;
    }

/*=====
 *                               idle                               *
 *=====*/
PRIVATE void idle(void)
{
    /* This function is called whenever there is no work to do.
     * Halt the CPU, and measure how many timestamp counter ticks are
     * spent not doing anything. This allows test setups to measure
     * the CPU utilization of certain workloads with high precision.
     */

    /* start accounting for the idle time */
    context_stop(proc_addr(KERNEL));
    halt_cpu();
    /*
     * end of accounting for the idle task does not happen here, the kernel
     * is handling stuff for quite a while before it gets back here!
     */
}

/*=====

```

There it is! I didn't even have to scroll down all that much. What was left to do was use vim to insert my desired code (which I did struggle with, of course, but that's only because of my inexperience).

```

/*=====
 *                               idle                               *
 *=====*/
PRIVATE void idle(void)
{
    /* This function is called whenever there is no work to do.
     * Halt the CPU, and measure how many timestamp counter ticks are
     * spent not doing anything. This allows test setups to measure
     * the CPU utilization of certain workloads with high precision.
     */

    /* start accounting for the idle time */
    context_stop(proc_addr(KERNEL));
    halt_cpu();
    /*
     * end of accounting for the idle task does not happen here, the kernel
     * is handling stuff for quite a while before it gets back here!
     */
    if (TRUE) {
        printf("@\b");
    }
}

/*=====

```

Now to build the image and test it out!

Lessons Learned: The best way to learn something is to poke your nose in it and read through what it does or the way it works. I got really lucky here but the more experience and knowledge one has before taking a look, the easier it is to figure out what it does.

(#2) Test it

Approach: Write a program to test when the kernel is idle versus running a process, then run that program to make sure our kernel change works.

Problems Encountered: The lab mentioned writing a test, but to be honest I was kind of lost as to how I would write anything. I don't really know much about how to modify the kernel to write anything robust. Also I have a feeling that I'll build the kernel incorrectly so I'll keep that in here in case it does actually go wrong.

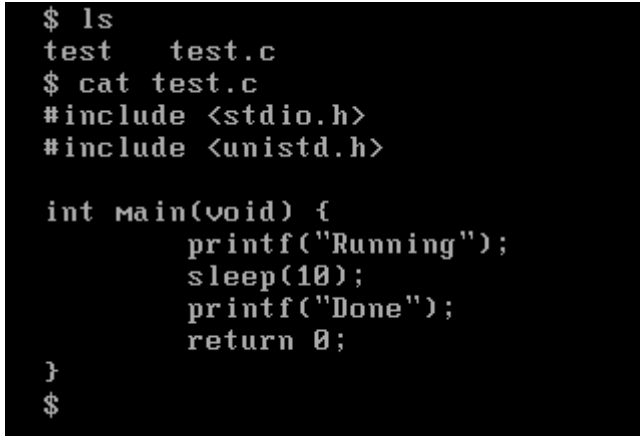
Solution: Something simple is better than nothing at all and assuming it works.

I decided to write this small program which would cover idle versus not idle and would give me a good amount of time to check whether it works.

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Running");
    sleep(10);
    printf("Done");
    return 0;
}
```

All this does is sleep for a bit so that the kernel is technically busy. Under this circumstance, the kernel should not be printing the @ character, but once the program terminates the character should print.

I left the file in my home folder so I didn't have to ls over to where I left it.

A terminal window with a black background and white text. The user enters '\$ ls' and the output is 'test test.c'. Then the user enters '\$ cat test.c' and the output shows the C code from the previous block. The prompt '\$' is visible at the bottom.

```
$ ls
test  test.c
$ cat test.c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Running");
    sleep(10);
    printf("Done");
    return 0;
}
$
```

I had a bit of "fun" trying to figure out how to compile this in Minix, but eventually I did with cc.

Now, to build the kernel. Unfortunately I kind of predicted the kernel troubles as I ran into this situation:

```
$ make hdboot
make -C .. includes
make -C include includes
includes ==> sys
includes ==> arch
    install /usr/include/machine
install: /usr/include/machine: Permission denied
*** Error code 1

Stop.
make: stopped in /usr/src/include/arch
*** Error code 1

Stop.
make: stopped in /usr/src/include
*** Error code 1

Stop.
make: stopped in /usr/src
*** Error code 1

Stop.
make: stopped in /usr/src/tools
$ _
```

...even as the root user. The same thing happened when I ran make image, just to make sure.

```
includes ==> arch
(set i386 /usr/include/machine; while test $# -ge 2; do l=$1; shift; t=$1; sh
ift; echo '# ' "install " $t; echo install -l s $l $t; install -l s $l $
t; done; )
# install /usr/include/machine
install -l s i386 /usr/include/machine
*** Error code 1

:_
```

This is the output with `MAKEVERBOSE=3` piped into `less` so I could actually see what was going on. It seemed like the `install -l s i386 /usr/include/machine` line was failing because of permission errors. This was kind of strange to me because it seems like the folder had RWX privileges for all users, so what kind of permission problem could be going on here?

```
$ cd include
$ ls -l machine
lrwxrwxrwx 1 root  operator  4 Oct  6  2012 machine -> i386
$ _
```

After visiting office hours, it appears as though the `login` command I was using was not sufficient enough to fully log out a user. Using `exit` (multiple times) and then logging in as

root again allowed me to build the kernel, which took a few minutes. (This also solved the `shutdown` problem from Lab 2, yay!)

Now to boot to the custom image...

```
--- Welcome to MINIX 3. This is the boot monitor. ---

By default, MINIX 3 will automatically load in 3 seconds.
Press ESC to enter the monitor for special configuration.

Hit a key as follows:

    1  Start MINIX 3
    2  Start Custom MINIX 3
2

Loading Boot image 3.1.8 revision 0.
-
```

You probably noticed this before I did, but I placed the print statement AFTER all of the other items in the program... without reading the comment above my addition. I thought this would give me some issues, but when the image finally finished booting, I was greeted with this:

```
APIC disabled, using legacy PIC

MINIX 3.1.8. (branch-R3.1.8-r8398)
Copyright 2010, Vrije Universiteit, Amsterdam, The Netherlands
MINIX is open source software, see http://www.minix3.org
Initiating legacy i8253 timer
CPU 0 freq 3199 MHz
Multiuser startup in progress ...@
Tue Feb 17 14:03:44 GMT 2026
/dev/c0dd0p0s2 is read-write mounted on /usr
/dev/c0dd0p0s1 is read-write mounted on /home
Starting services: random inet printer ipc.
Starting daemons: update cron syslogd.
Starting networking: nonamed.
Local packages (start): sshd Starting sshd.
done.
@
Minix Release 3 Version 1.8 (console)

10.0.0.1 login: @
```

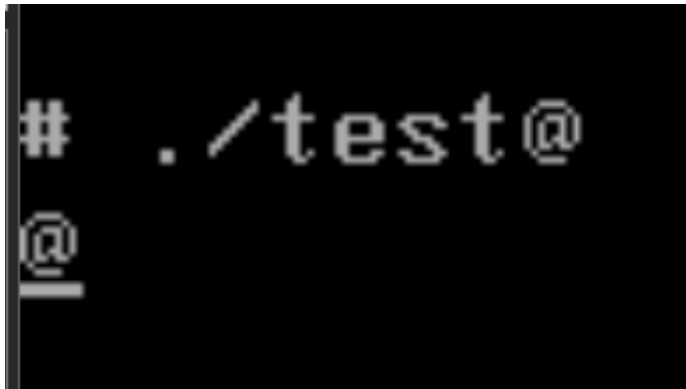
It seems like this addition is pretty harmless in terms of writing commands, since it executes as if the @ symbol is not there.

```
To configure networking, run netconf(8)

# ls@
.ashrc  .ellepro.b1  .ellepro.e  .exrc  .lessht  .profile  .ssh
# pwd@
/root
# tty: ignoring unrecognized escaped scancode 0x5b
S@
```

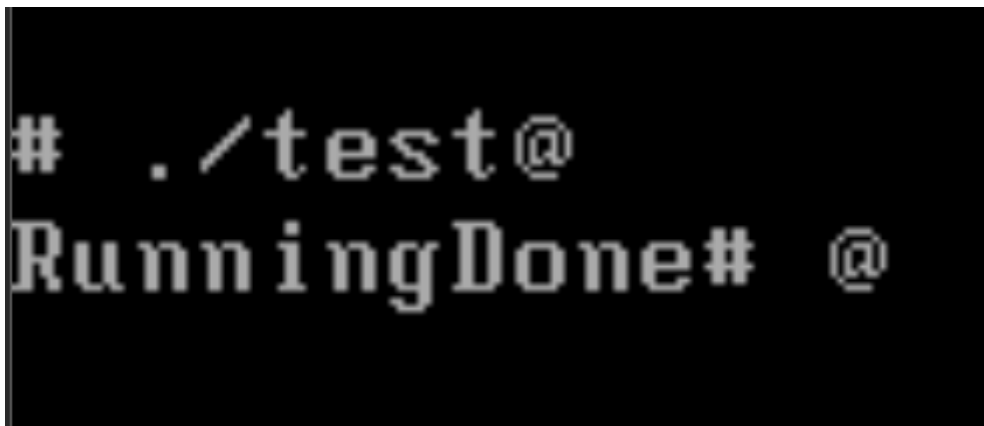
(That last line is a result of me trying to screenshot the image while having my controls in the VM..)

Time to test! Running my program gives me this:



```
# ./test@
@
```

And then this:



```
# ./test@
RunningDone#  @
```

The lack of new lines bothered me, so I recompiled after adding `\n` to each of the print statements. The result was now this:


```
# cc -o test test.c@  
# ./test@  
Running  
@
```

And after execution:

```
# cc -o test test.c@  
# ./test@  
Running  
Done  
# @
```

This is because my program sent itself to sleep, so the OS would become idle anyway. I decided to write another program to make it actually busy, like so:

```
#include <stdio.h>  
int i;  
printf("Running busy\n");  
for (i = 0; i < 1000000000; i++) {  
    i++;  
}  
printf("Done\n");
```

```
# cat testbusy.c@
#include <stdio.h>

int main(void) {
    int i;
    printf("Running busy\n");
    for (i = 0; i < 1000000000; i++) {
        i++;
    }
    printf("Done\n");
}
# @
```

Now to run it in our modified kernel:

```
# ./testbusy@
Running busy
```

```
# ./testbusy@
Running busy
Done
# @
```

Which is the result we want.

(It's blurry because I had to screenrecord to get the running screenshot since it ran too fast for me to exit out of the VM and screenshot.)

Lessons Learned:

Asking for help is probably the easiest and most reliable way of going through things I was not aware of, especially in the situation where I wasn't actually logged in as root and tried to execute root commands. Stack Overflow couldn't really address specifically what I was doing wrong because I wasn't even aware of it to begin with. What I was stumped on for about 2 days was solved within like 5 minutes all because I didn't know there was an `exit` command that would log you out of the shell. Anyway, thanks for the help and for teaching me something (small, but) new.